

VoteChain: A Secure Blockchain-Based Online Voting System

Mohmaed Wassim Dobbi

Department Of Computer Engineering
Istanbul Kultur University
Istanbul, Turkey
1900000316@stu.iku.edu.tr

Mohamed Sayed

Department Of Computer Engineering
Istanbul Kultur University
Istanbul, Turkey
2300000497@stu.iku.edu.tr

Juman Mazen Hudhud Mughrbı

Department of Computer Engineering
Istanbul Kultur University
Istanbul, Turkey
2200000639@stu.iku.edu.tr

Sara Morei Bouzou

Department Of Computer Engineering
Istanbul Kultur University
Istanbul, Turkey
2200007137@stu.iku.edu.tr

Omar Habib

Department Of Computer Engineering
Istanbul Kultur University
Istanbul, Turkey
2200000372@stu.iku.edu.tr

Yahia Okba

Department Of Computer Engineering
Istanbul Kultur University
Istanbul, Turkey
2200001248@stu.iku.edu.tr

Ahmed Abdulhakim Ali Alajnaf

Department Of Computer Engineering
Istanbul Kultur University
Istanbul, Turkey
2200002368@stu.iku.edu.tr

Abstract—VoteChain addresses key challenges in online voting through blockchain technology, creating a secure, transparent platform that maintains voter privacy while ensuring verifiable results. Our system leverages Ethereum smart contracts and IPFS decentralized storage to implement immutable vote recording, and automated tallying. The architecture combines secure government ID verification with blockchain’s immutability while minimizing on-chain data to preserve privacy. Smart contracts enforce electoral rules through tamper-proof code that prevents double voting and ensures only eligible voters participate. Through rigorous and strict testing and security analysis, VoteChain demonstrates the potential to transform electoral processes to enhance trust in digital democracy while maintaining compliance with electoral regulations.

Index Terms—blockchain, smart contracts, e-voting, decentralized identity, IPFS, Ethereum, voting system

I. INTRODUCTION

Traditional voting systems face numerous limitations including high costs, accessibility barriers, and vulnerability to fraud. Paper-based voting requires significant resources and presents challenges for voters with mobility restrictions, while manual counting introduces potential for error or manipulation [3].

Online voting systems offer enhanced accessibility and reduced costs but continue to face critical challenges related to security, transparency, and privacy. Centralized digital systems remain vulnerable to attacks that could compromise entire elections without leaving visible evidence [4].

Blockchain technology provides a promising foundation for addressing these challenges through immutable, transparent,

and decentralized records [5]. VoteChain leverages these properties to create a secure online voting platform that preserves voter privacy while ensuring transparent and verifiable results.

Unlike traditional centralized voting systems, VoteChain distributes trust across a network of nodes, making it resistant to single points of failure and manipulation. The blockchain’s consensus mechanism ensures that votes are verified by multiple independent validators before being permanently recorded, creating a transparent yet tamper-resistant electoral record.

This paper details the design and implementation of VoteChain, highlighting its innovative approach to voter authentication, vote casting, and result verification. Our research potential contributes to enhance democratic processes.

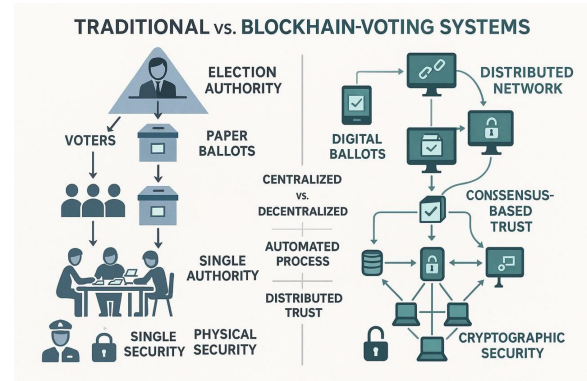


Fig. 1: Difference between traditional and blockchain based voting process.

II. RELATED WORK

A. Previous Blockchain Voting Systems

Early blockchain voting implementations focused primarily on immutability with less emphasis on privacy requirements. Commercial projects like FollowMyVote and Voatz pioneered blockchain integration for remote digital voting, emphasizing verifiability and immutability of records. However, as noted by Park et al. [8], these implementations often relied on centralized identity management or lacked robust privacy guarantees. The Voatz platform employed biometric verification combined with blockchain record-keeping but maintained centralized control over voter authentication, creating potential vulnerabilities that contradicted blockchain's decentralized promise.

Academic research by Khoury et al. [9] identified several critical limitations in first-generation blockchain voting systems: inadequate voter privacy protection, susceptibility to Sybil attacks, and challenges in scalability. These limitations highlighted the need for more sophisticated approaches to protect voter privacy while maintaining system integrity.

B. Identity Management and VoteChain's Approach

A fundamental challenge in digital voting is ensuring that only eligible voters participate while maintaining privacy. Decentralized identity (DID) technologies offer promising solutions to this challenge. As described by Reed et al. [10], DIDs enable self-sovereign identity models where individuals control their digital identities without relying on centralized authorities.

Wang et al. [11] explored the integration of verifiable credentials with blockchain voting, proposing a system where government-issued credentials could be verified without revealing the voter's full identity. This approach addressed the "eligibility vs. privacy" dilemma that has challenged digital voting implementations.

VoteChain builds upon these foundations by implementing a practical approach that prioritizes security, privacy, and usability. The system implements secure verification using government-issued credentials while storing only the minimum necessary verification data. The architecture separates concerns between on-chain and off-chain components, with sensitive operations occurring off-chain to preserve privacy and reduce computational costs, following best practices identified by Garay et al. [13].

III. SYSTEM DESIGN AND ARCHITECTURE

A. System Overview

VoteChain employs a modular, multi-layered architecture designed to ensure security, privacy, and verifiability throughout the voting process. This architecture follows the separation of concerns principle identified by Buterin [15] as essential for complex blockchain applications.

The system consists of four primary layers:

- **Frontend Layer:** Provides the user interface for voters and administrators using React.js and Vite

- **Backend Layer:** Bridges frontend and blockchain, handling identity verification and communication with decentralized storage
- **Blockchain Layer:** Consists of Ethereum-based smart contracts that manage voter registration, election configuration, and vote tallying
- **Decentralized Storage Layer:** Utilizes IPFS [2] to store election metadata, candidate information, and results

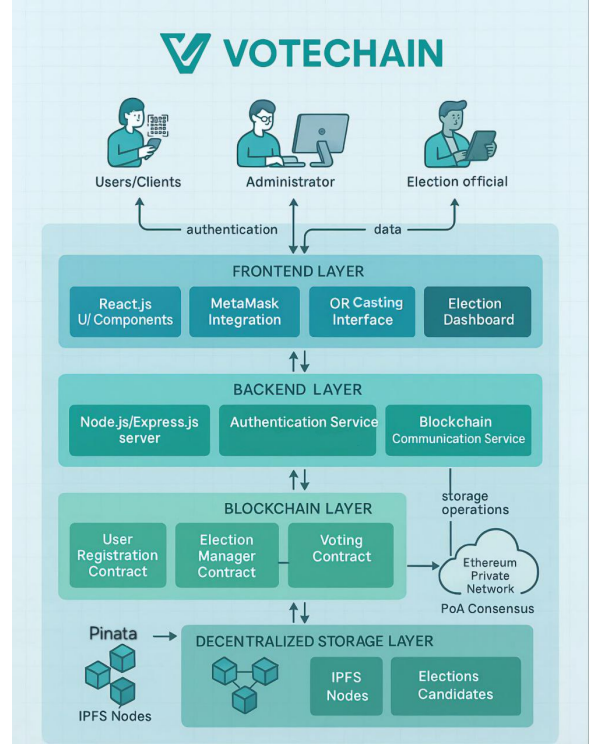


Fig. 2: VoteChain's deployment architecture illustrating inter-connections between frontend components, backend services, blockchain smart contracts, and decentralized storage.

B. Smart Contract Architecture

VoteChain's blockchain layer consists of several interconnected smart contracts that implement the core logic of the voting process. These contracts provide programmatic enforcement of voting rules without requiring trust in central authorities [19].

The **User Registration Contract** manages identity verification and user registration, establishing a secure binding between a unique identifier (UUID) and the voter's blockchain wallet address through bidirectional mappings. This prevents identity spoofing and multiple registrations by the same individual.

The **Election Manager Contract** handles administrative functions including election creation, candidate registration, and timeline management. Administrators can create elections, add candidates, set voting periods, and manage parameters through this contract's functions.

The **Voting Contract** manages the vote casting process and implements comprehensive validation to prevent electoral fraud. This contract verifies multiple conditions before recording a vote: the voter must be registered, must not have voted previously in the election, the election must be active, and the selected candidate must be valid.

The **Results Contract** handles vote tallying and winner determination, with controlled visibility and secure computation of final outcomes. This contract records votes as they are cast and provides functions for calculating election results once voting has concluded.

C. Authentication and Identity Verification

VoteChain implements a secure authentication system using government-issued credentials. This approach ensures that voters can authenticate securely while proving essential eligibility attributes like nationality and age without exposing unnecessary personal information.

The authentication process follows a simple flow that balances security with usability. Voters authenticate using their government-issued credentials, and the system verifies only the minimum necessary attributes to determine voting eligibility. After successful authentication, voters connect their blockchain wallet to complete the process, creating a secure link between verified identity and voting transactions.

IV. IMPLEMENTATION

A. Smart Contract Implementation

Smart contracts form the foundation of trust in the VoteChain system, establishing immutable rules for all voting operations. These self-executing contracts with terms directly written into code create a trustless environment where electoral rules are enforced consistently [23].

The User Registration Contract (Algorithm 1) implements a secure voter registration process that establishes a bidirectional mapping between a user's unique identifier (UUID) and their blockchain wallet address. This design prevents multiple identities from being associated with a single wallet address and vice versa, addressing the Sybil attack vulnerability identified in early blockchain voting systems [9].

Listing 1: Algorithm 1: Smart Contract: Registering User

```

1: Require: Initialization of parameters
2: Initialize voter_id = this voter_id
3: Initialize voter_name = this voter_name
4: Func (Register Voter)
5: Input: uuid
6: Require: bytes(uuid).length > 0
7: Require: !isRegistered[uuid]
8: Require: bytes(walletToUuid[msg.sender]).length == 0
9: uuidToWallet[uuid] = msg.sender
10: walletToUuid[msg.sender] = uuid
11: isRegistered[uuid] = true
12: emit UserRegistered event
13: End Func
14: End Smart Contract

```

The Voting Contract (Algorithm 2) manages the vote casting process with comprehensive validation. Before recording a vote, the contract verifies that the sender is registered, that they haven't previously voted in this election, that the election is active, and that the selected candidate is valid. This multi-layered validation approach prevents common voting attacks including double voting and voting for non-existent candidates.

Listing 2: Algorithm 2: Smart Contract: Casting Vote

```

1: Require: Initialization of parameters
2: Initialize uuid=
userReg.walletToUuid(msg.sender)
3: Func (Vote)
4: Input: electionId, candidate
5: Require: bytes(uuid).length > 0
6: Require: !hasVoted[electionId][uuid]
7: Require:
!walletHasVoted[electionId][msg.sender]
8: Require:
electionMgr.isElectionActive(electionId)
9: Require:
electionMgr.isCandidateInElection(
electionId,
candidate)
10: hasVoted[electionId][uuid] = true
11: walletHasVoted[electionId][msg.sender]
= true
12: results.recordVote(electionId,
candidate)
13: emit VoteCasted event
14: End Func
15: End Smart Contract

```

The Results Contract (Algorithm 3) maintains an accurate tally of votes. When recording a vote, the contract first checks if the candidate exists in its records for the specified election. If not, it adds the candidate to the array of election candidates. It then increments the vote count for the candidate and the total vote count for the election, enabling dynamic candidate addition and efficient vote tracking.

Listing 3: Algorithm 3: Smart Contract: Recording Vote

```

1: Require: Initialization of parameters
2: Initialize electionId = this electionId
3: Initialize candidate = this candidate
4: Func (Record Vote)
5: Input: electionId, candidate
6: If !candidateExists[electionId][
candidate]
7: then
electionCandidates[electionId].push(
candidate)
8: candidateExists[electionId][candidate] =
true
9: candidateVotes[electionId][candidate]++
10: totalVotes[electionId]++
11: emit VoteRecorded event
12: End Func
13: End Smart Contract

```

The winner determination function (Algorithm 4) implements an efficient process for identifying the candidate with the most votes. The function verifies that results are visible and at least one vote has been cast, then iterates through all candidates to find the one with the highest total. Access control mechanisms ensure that results are only visible when appropriate, allowing for controlled result release after voting concludes.

Listing 4: Algorithm 4: Smart Contract: Determining Winner

```

1: Require: Initialization of parameters
2: Initialize electionId = this electionId
3: Func (Get Winner)
4: Input: electionId
5: Require: resultsVisible[electionId]
6: Require: totalVotes[electionId] > 0
7: Initialize highest = 0
8: Initialize winner = ""
9: Initialize cand = electionCandidates[
  electionId]
10: For i = 0 to cand.length
11: Initialize v = candidateVotes[
  electionId][cand[i]]
12: If v > highest
13: then highest = v
14: winner = cand[i]
15: Return winner
16: End Func
17: End Smart Contract

```

B. Front End and Decentralized Storage Implementation

The frontend implementation utilizes React.js with MetaMask integration for blockchain interaction. Web3.js and Ethers.js libraries facilitate communication with deployed smart contracts. The user interface guides voters through registration, verification, vote casting, and result viewing processes with security considerations driving significant aspects of the design.

VoteChain incorporates IPFS for decentralized storage of electoral data. Three logical directories handle different aspects of electoral information: Elections, Candidates, and Results. This decentralized storage complements the blockchain's immutability with flexible content management. While the blockchain records votes and ensures their integrity, IPFS provides efficient storage for larger data objects that don't require transaction-based consensus.

V. SECURITY AND PRIVACY ANALYSIS

A. Threat Model and Security Considerations

Our threat model considers various potential attacks that could compromise the integrity, confidentiality, or availability of the voting process [?].

1) *Identity and Authentication Threats:* VoteChain addresses identity challenges:

Sybil Attack Prevention: The bidirectional mapping between unique identifiers (UUID) and wallet addresses prevents a single individual from creating multiple voting identities.

2) *Double Voting Prevention:* VoteChain implements comprehensive double-voting prevention through multiple validation layers:

- UUID-based voting status verification
- Wallet address-based voting status verification
- Cross-contract validation with the Election Manager

3) *Network-Level Attacks:* VoteChain's distributed architecture provides inherent resistance to DoS attacks compared to centralized voting servers. The system employs multiple defensive layers including distributed blockchain nodes, IPFS storage.

B. Privacy Mechanisms

Privacy in voting systems must balance voter anonymity with eligibility verification. VoteChain implements several privacy-preserving mechanisms:

1) *Minimal Disclosure Authentication:* The system allows voters to prove essential attributes (nationality, age) without revealing their full identity through:

- Selective disclosure of credential attributes

2) *Vote Privacy:* Once authenticated, VoteChain ensures vote privacy through transaction pseudonymity where votes are linked to wallet addresses rather than real identities. The system implements temporal decoupling between authentication and voting phases to prevent correlation through timing analysis.

VI. DISCUSSION AND LIMITATIONS

A. Security and Trust Considerations

The VoteChain implementation demonstrates that Ethereum's cryptographic guarantees can effectively create a tamper-proof voting record. The security model shifts from trust in central authorities to trust in code and cryptography, representing what De Filippi and Wright [27] characterize as a move from "institutional trust" to "algorithmic trust."

B. Performance and Scalability Trade-offs

Deploying VoteChain on a private Ethereum network with Proof-of-Authority (PoA) consensus balanced performance with security needs. This approach provided control over validator selection and block times, making the system more responsive than public blockchain implementations. In our testing, the system demonstrated stable performance with thousands of simulated voters, maintaining consistent transaction confirmation times of approximately 5-10 seconds even under load.

This configuration sacrifices some degree of decentralization, which may be acceptable for regulated election environments but represents a trade-off between performance and the trust-minimization benefits of public blockchains. The validator set in our implementation consisted of seven nodes operated by different administrative entities, creating a balance between operational efficiency and distributed trust.

C. Limitations and Future Directions

Several limitations remain that warrant consideration. The balance between privacy and transparency presents an ongoing challenge in blockchain voting systems. Even with minimal credential disclosure, the system must carefully navigate between the transparency benefits of blockchain and the privacy requirements of democratic elections.

The dependency on external identity systems introduces potential centralization points in an otherwise decentralized architecture. While the current implementation minimizes data shared between government verification systems and the voting platform, this integration still creates a trust dependency. Emerging standards like the Decentralized Identifiers specification provide promising frameworks for reducing this dependency.

Despite interface simplifications, the system still requires basic technological literacy and access to compatible devices, which may exclude certain populations. Future implementations could explore offline voting options with blockchain reconciliation, public voting kiosks, or delegated voting mechanisms.

VII. CONCLUSION

VoteChain demonstrates the practical application of blockchain technology in enhancing the transparency, security, and integrity of online voting systems. Through a private Ethereum network with Proof-of-Authority consensus, the system successfully addresses key challenges in digital voting while providing a framework for secure and verifiable electoral processes.

The system design incorporates multiple integrated layers that work together to create a comprehensive voting platform. This architecture addresses fundamental requirements for democratic voting: eligibility verification, vote secrecy, tamper resistance, and result verifiability.

Our implementation demonstrates innovative approaches to integrating blockchain with identity verification, showing how decentralized technologies can be practically applied to governance challenges. The smart contract architecture establishes a pattern for implementing complex business logic on blockchain platforms while maintaining gas efficiency and security.

Test results confirm that the system can handle realistic electoral scales with acceptable performance while maintaining the security guarantees expected from blockchain technology. Despite these achievements, challenges remain in scaling blockchain voting for national elections, reducing technical barriers to participation, and addressing the inherent trade-offs between transparency and privacy.

REFERENCES

- [1] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum White Paper, 2013. [Online]. Available: <https://ethereum.org/en/whitepaper/>
- [2] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," 2014. [Online]. Available: <https://ipfs.io/ipfs/QmR7GSQM93C5eAg6a6RxmRzTGPfb8fZb8TH2RA1XWzWQk>
- [3] D. Jefferson, A. D. Rubin, B. Simons, and D. Wagner, "A Security Analysis of the Secure Electronic Registration and Voting Experiment (SERVE)," New York Times, 2004.
- [4] J. A. Halderman et al., "Security Analysis of the Estonian Internet Voting System," in Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, 2014, pp. 703-715.
- [5] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [6] P. De Filippi and A. Wright, "Blockchain and the Law: The Rule of Code," Harvard University Press, 2018.
- [7] P. Boucher, "What if blockchain technology revolutionised voting?," European Parliamentary Research Service, 2016.
- [8] S. Park, M. Specter, N. Narula, and R. L. Rivest, "Going from Bad to Worse: From Internet Voting to Blockchain Voting," Journal of Cybersecurity, vol. 7, no. 1, 2021.
- [9] D. Khoury, E. F. Kfoury, A. Kassem, and H. Harb, "Decentralized Voting Platform Based on Ethereum Blockchain," in 2018 IEEE International Multidisciplinary Conference on Engineering Technology (IMCET), 2018.
- [10] D. Reed et al., "Decentralized Identifiers (DIDs) v1.0," W3C, 2021. [Online]. Available: <https://www.w3.org/TR/did-core/>
- [11] F. Wang, P. De Filippi, and A. Pentland, "A Fair Governance Model for Blockchain," arXiv preprint arXiv:2010.07377, 2020.
- [12] P. Dunphy and F. A. P. Petitcolas, "A First Look at Identity Management Schemes on the Blockchain," IEEE Security & Privacy, vol. 16, no. 4, pp. 20-29, 2018.
- [13] J. A. Garay, A. Kiayias, and N. Leonardos, "The Bitcoin Backbone Protocol: Analysis and Applications," in Advances in Cryptology - EUROCRYPT 2015, 2015, pp. 281-310.
- [14] S. Heiberg, P. Laud, and J. Willemson, "The Application of I-Voting for Estonian Parliamentary Elections of 2011," in E-Voting and Identity, 2012, pp. 208-223.
- [15] V. Buterin, "The Meaning of Decentralization," Medium, 2017. [Online]. Available: <https://medium.com/@VitalikButerin/the-meaning-of-decentralization-a0c92b76a274>
- [16] J. Nielsen, "Usability 101: Introduction to Usability," Nielsen Norman Group, 2012. [Online]. Available: <https://www.nngroup.com/articles/usability-101-introduction-to-usability/>
- [17] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Project Yellow Paper, 2014.
- [18] P. A. Grassi et al., "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, 2017.
- [19] A. M. Antonopoulos and G. Wood, "Mastering Ethereum: Building Smart Contracts and DApps," O'Reilly Media, 2018.
- [20] IEEE, "IEEE Standard for the Format of Public Key Certificates," IEEE Std 1363.3-2013, 2013.
- [21] J. Nielsen and R. Budiu, "Mobile Usability," Nielsen Norman Group, 2012.
- [22] D. A. Gritzalis, "Principles and Requirements for Security in Electronic Voting Systems," Journal of Information Warfare, vol. 1, no. 2, pp. 41-60, 2002.
- [23] N. Szabo, "Smart Contracts: Building Blocks for Digital Markets," 1996.
- [24] X. Xu et al., "The Blockchain as a Software Connector," in 2016 13th Working IEEE/IFIP Conference on Software Architecture (WICSA), 2016, pp. 182-191.
- [25] European Parliament and Council, "General Data Protection Regulation (GDPR)," Regulation (EU) 2016/679, 2016.
- [26] T. Moura and A. Gomes, "Blockchain Voting and Its Effects on Election Transparency and Voter Confidence," in Proceedings of the 18th Annual International Conference on Digital Government Research, 2017, pp. 574-575.
- [27] P. De Filippi and A. Wright, "Decentralized Blockchain Technology and the Rise of Lex Cryptographia," SSRN Electronic Journal, 2015.
- [28] Q. Xu, K. M. M. Aung, Y. Zhu, and K. L. Yong, "A Blockchain-Based Storage System for Data Analytics in the Internet of Things," in New Advances in the Internet of Things, 2018, pp. 119-138.
- [29] J. Nielsen, "Ten Usability Heuristics," Nielsen Norman Group, 1994.
- [30] R. Osgood, "The Future of Democracy: Blockchain Voting Systems," SSRN Electronic Journal, 2016.